

Vehicle Service Management System

Installation and Setup Guide

Prerequisites

Before setting up the application, ensure you have the following software installed:

Required Software

Backend Requirements:

- .NET SDK 9.0 or higher
- SQL Server 2019 or higher (Express/Developer/Enterprise Edition)
- Entity Framework Core Tools

Frontend Requirements:

- Node.js 18.x or higher
- npm (Node Package Manager)
- Angular CLI (latest version)

Verify Installation

Open your terminal/command prompt and run the following commands to verify installations:

```
npm install -g @angular/cli
```

```
dotnet --version
```

```
node --version
```

```
ng version
```

Setup and Installation

Step 1: Clone the Repository

Open your terminal and execute:

```
git clone <repository-url>
```

```
cd VehicleServiceManagement.API
```

Step 2: Backend Setup

2.1 Configure Database Connection

Locate and open the appsettings.json file in the API project root directory.

For Windows Authentication (Trusted Connection):

```
{
  "ConnectionStrings": {
    "DefaultConnection":
    "Server=(localdb)\\MSSQLLocalDB;Database=VehicleServiceManagementDB;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

2.2 Configure JWT Settings

Add the following JWT configuration in appsettings.json:

```
"JwtSettings": {
  "Secret":
  "YourSuperSecretKeyThatIsAtLeast32CharactersLong!VehicleServiceManagement2024",
  "Issuer": "VehicleServiceAPI",
  "Audience": "VehicleServiceClient",
  "ExpiryInMinutes": 60
},
"AllowedHosts": "*"
}
```

Important: Replace the Key value with a strong, unique secret key of at least 32 characters.

2.3 Configure Email Settings (Optional)

For email notifications and OTP functionality, configure SMTP settings:

```
{
  "SmtpSettings": {
    "Host": "smtp.gmail.com",
    "Port": 587,
    "EnableSsl": true,
    "UserName": "vehicleservice.notify@gmail.com",
    "Password": "sonpvelgrblckhib",
    "FromEmail": "vehicleservice.notify@gmail.com",
    "FromName": "Chubb Vehicle Management System"
  }
}
```

Note: For Gmail, you need to generate an App Password from your Google Account settings.

2.4 Run Database Migrations

Navigate to the API project directory and execute:

```
cd VehicleServiceManagement.API
```

```
dotnet ef database update
```

If Entity Framework Tools are not installed:

```
dotnet tool install --global dotnet-ef
```

2.5 Seed Initial Data

The database seeder runs automatically on the first application startup and creates:

- Default admin user
 - Email: admin@example.com
 - Password: Admin@123!
- Sample service categories
- Sample parts inventory

Step 3: Frontend Setup

3.1 Install Dependencies

Navigate to the UI project directory and install npm packages:

```
cd VehicleServiceManagement.UI
```

```
npm install
```

3.2 Configure API URL

Open src/environments/environment.ts and update the API URL:

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:5081/api'  
};
```

Ensure the port number matches your backend configuration.

Running the Application

Start the Backend API

Open a terminal, navigate to the API project, and run:

```
cd VehicleServiceManagement.API
```

```
dotnet run
```

Start the Frontend Application

Open a new terminal window, navigate to the UI project, and run:

```
cd VehicleServiceManagement.UI
```

```
ng serve
```

The Angular application will compile and be available at:

- <http://localhost:4200>

Open your web browser and navigate to this URL to access the application.

Default Credentials

Use these credentials for initial login:

Role	Email	Password
Admin	admin@example.com	Admin@123!

Additional users can be created through:

- Registration flow (requires admin approval for ServiceManager and Technician roles)
- Admin panel (Admin users only)

Authentication and Authorization

How Authentication Works

The application uses JWT (JSON Web Token) based authentication:

1. User submits credentials to `/api/auth/login` endpoint
2. Server validates credentials against the database
3. If valid, server generates a JWT token containing user claims (userId, email, role)
4. Frontend stores the token securely in browser `localStorage`
5. All subsequent API requests include the token in the Authorization header:
Authorization: Bearer <token>
6. Backend middleware validates the token and extracts user information for authorization

Role-Based Access Control

Routes and API endpoints are protected by user roles to ensure proper access control.

Backend Protection (Controller Level):

```
[Authorize(Roles = "Admin,ServiceManager")]

public class DashboardController : ControllerBase
{
    // Only Admin and ServiceManager can access
}
```

Backend Protection (Action Level):

```
[Authorize(Roles = "Admin")]

public async Task<IActionResult> ApproveUser(int id)
{
    // Only Admin can approve users
}
```

Frontend Protection (Route Guard):

```
{
  path: 'dashboard',
  component: DashboardComponent,
  canActivate: [AuthGuard],
  data: { roles: ['Admin', 'ServiceManager'] }
}
```

User Roles and Permissions

Role	Access Level	Key Permissions
Admin	Full system access	Manage users, approve registrations, configure system settings, view all data
ServiceManager	Operations management	Assign technicians, manage service categories, generate bills, view reports
Technician	Assigned tasks only	View assigned tasks, update task status, add service remarks
Customer	Own data only	Book services, view own vehicles, track service requests, make payments

API Documentation

Swagger UI

When running in development mode, interactive API documentation is available at:

URL: `http://localhost:5081/swagger`

Swagger UI provides:

- Complete list of all API endpoints
- Request/response schemas
- Interactive testing capability
- Authentication testing

Using Swagger for Testing

1. Navigate to the Swagger URL
2. Click "Authorize" button
3. Enter the JWT token in format: Bearer <your-token>
4. You can now test protected endpoints

Project Structure

Backend Project Structure

VehicleServiceManagement.API/

```
├── Controllers/    # API endpoint controllers
├── Models/        # Entity and DTO models
├── Services/      # Business logic services
├── Data/          # DbContext and repositories
├── Middleware/    # Custom middleware (auth, exception handling)
├── Helpers/       # Utility classes
└── appsettings.json # Configuration file
```

Frontend Project Structure

VehicleServiceManagement.UI/

```
├── src/
|   ├── app/
|   |   ├── core/    # Auth, guards, interceptors
|   |   └── shared/  # Reusable components
```

```
| | |— features/    # Feature modules
| | |— models/     # TypeScript interfaces
| |— environments/ # Environment configurations
```

Logging

The application logs important events and errors. Check:

- Backend logs: Console output where dotnet run is executed
- Frontend logs: Browser console (F12 → Console tab)